

Grupo ARCOS

uc3m | Universidad **Carlos III** de Madrid

Tema 3: Fundamentos de la programación en ensamblador (IV) **Estructura de Computadores**

Grado en Ingeniería Informática
Grado en Matemática aplicada y Computación
Doble Grado en Ingeniería Informática y Administración de Empresas



Contenidos

1. Fundamentos básicos de la programación en ensamblador
2. Ensamblador del RISC-V 32, modelo de memoria y representación de datos
3. Formato de las instrucciones y modos de direccionamiento
4. Coma flotante en ensamblador
5. Llamadas a procedimientos y uso de la pila

Banco de registros (coma flotante)

Nombre ABI	Nombre	Uso
ft0...ft7	f0 ... f7	Temporales (como los t...)
fs0...fs1	f8 ... f9	Se guardan (como los s...)
fa0...fa1	f10 ... f11	Argumentos/retorno (como los a...)
fa2...fa7	f12 ... f17	Argumentos (como los a...)
fs2...fs11	f18 ... f27	Se guardan (como los s...)
ft8...ft11	f28 ... f31	Temporales (como los t...)

- ▶ Hay 32 registros
- ▶ El registro ft0 no tiene su valor a 0
- ▶ En la extensión de simple precisión los registros son de 32 bits (4 bytes)
- ▶ En la extensión de doble precisión los registros son de 64 bits (8 bytes) y pueden almacenar:
 - ▶ Valores de simple precisión en los 32 bits inferiores del registro
 - ▶ Valores de do le precisión en los 64 bits del registro

Operaciones aritméticas

- ▶ Copia de registros (.s .d):

- ▶ `fmv.s rd rs` # $rd = rs$

- ▶ Operaciones aritmética comunes (.s .d):

- ▶ `fadd.s rd rs1 rs2` # $rd = rs1 + rs2$

- ▶ `fsub.s rd rs1 rs2` # $rd = rs1 - rs2$

- ▶ `fmul.s rd rs1 rs2` # $rd = rs1 * rs2$

- ▶ `fdiv.s rd rs1 rs2` # $rd = rs1 / rs2$

- ▶ `fmin.s rd rs1 rs2` # $rd = \min(rs1, rs2)$

- ▶ `fmax.s rd rs1 rs2` # $rd = \max(rs1, rs2)$

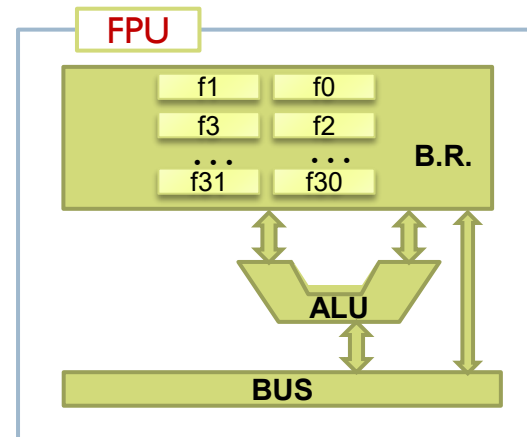
- ▶ `fsqrt.s rd rs1` # $rd = \sqrt{rs1}$

- ▶ `fmadd.s rd rs1 rs2 rs3` # $rd = rs1 \times rs2 + rs3$

- ▶ `fmsub.s rd rs1 rs2 rs3` # $rd = rs1 \times rs2 - rs3$

- ▶ `fabs.s rd rs` # $rd = |rs|$

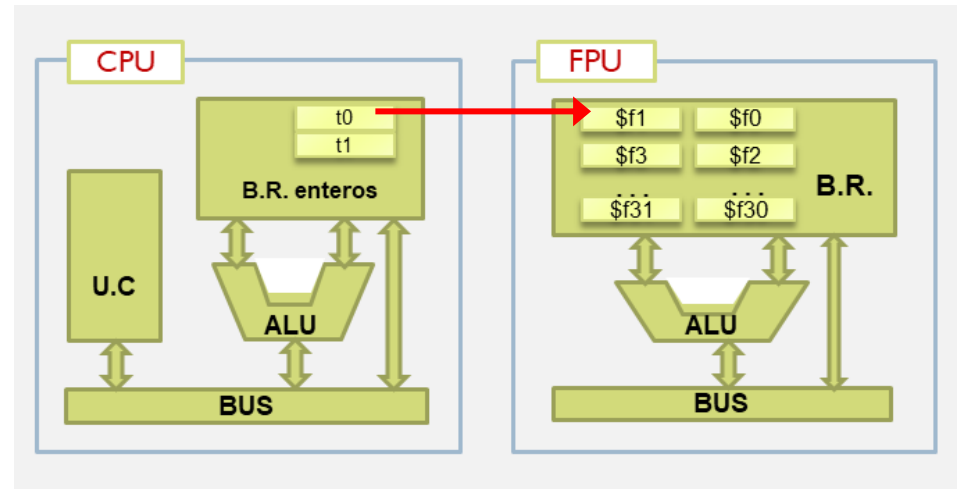
- ▶ `fneg.s rd rs` # $rd = -rs$



Operaciones de copia (registros enteros <-> registros coma flotante)

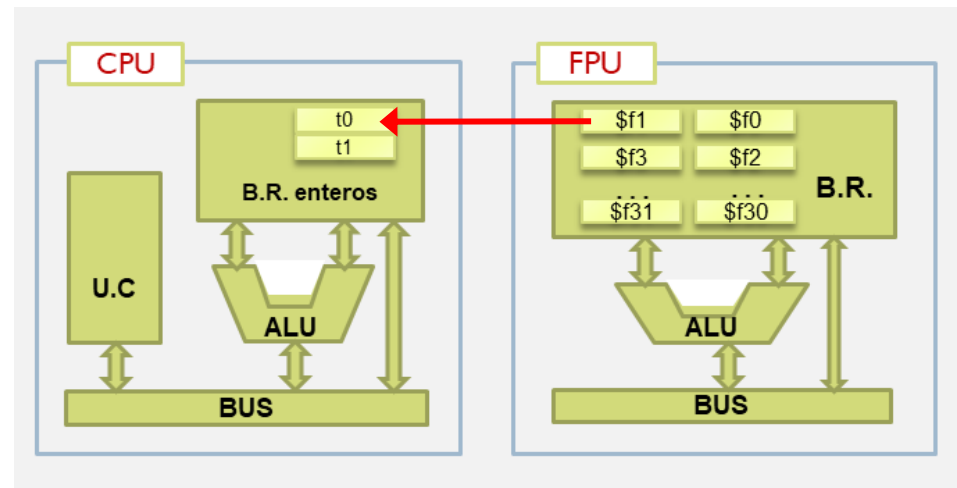
`fmv.w.x rd rs`

- Copia desde reg. entero `rs` a reg. flotante `rd` (*single precision*)



`fmv.x.w rd rs`

- Copia desde reg. flotante `rs` (*single precision*) a reg. entero `rd`



Operaciones de conversión (1 / 3)

entero <-> simple precisión

▶ `fcvt.w.s rd, rs`

- ▶ Convierte de simple precisión (valor en registro flotante rs) a entero de 32 bits **con** signo (registro entero rd).

▶ `fcvt.wu.s rd, rs`

- ▶ Convierte de simple precisión (valor en registro flotante rs) a entero de 32 bits **sin** signo (registro entero rd).

▶ `fcvt.s.w rd, rs`

- ▶ Convierte de entero de 32 bits **con** signo (valor en registro entero rs) a simple precisión (registro flotante rd).

▶ `fcvt.s.wu rd, rs`

- ▶ Convierte de entero de 32 bits **sin** signo (valor en registro entero rs) a simple precisión (registro flotante rd).

Operaciones de conversión (2/3)

entero <-> doble precisión

► `fcvt.w.d rd, rs`

- Convierte de doble precisión (valor en registro flotante rs) a entero de 32 bits **con** signo (registro entero rd).

► `fcvt.wu.d rd, rs`

- Convierte de doble precisión (valor en registro flotante rs) a entero de 32 bits **sin** signo (registro entero rd).

► `fcvt.d.w rd, rs`

- Convierte de entero de 32 bits **con** signo (valor en registro entero rs) a doble precisión (registro flotante rd).

► `fcvt.d.wu rd, rs1`

- Convierte de entero de 32 bits **sin** signo (valor en registro entero rs) a doble precisión (registro flotante rd).

Operaciones de conversión (3/3)

doble precisión $\leftrightarrow$ simple precisión

► `fcvt.s.d rd, rs1`

- Convierte de doble precisión (valor en registro flotante rs) a simple precisión (registro flotante rd).

► `fcvt.d.s rd, rs`

- Convierte de simple precisión (valor en registro flotante rs) a doble precisión (registro flotante rd).

Clasificación de números en coma flotante

- ▶ `fclass.s rd, rs1` (simple precisión)
- ▶ `fclass.d rd, rs1` (doble precisión)
- ▶ Escribe en `rd` el tipo de número de coma flotante del registro `rs1`:

Valor en rd	Significado
0	-Inf
1	Número normalizado negativo
2	Número no normalizado negativo
3	-0
4	+0
5	Numero no normalizado positivo
6	Número normalizado positivo
7	+Inf
8	NaN (no silencioso)
9	NaN (silencioso)

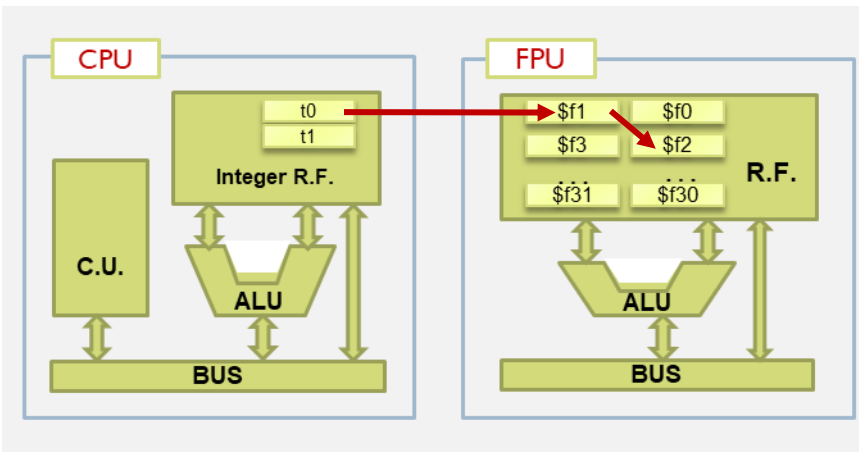
Ejemplo

```
float PI    = 3,1415;
int  radio = 4;
float length;

length = PI * radio;
```

```
.text
main:
```

```
li t0, 0x40490E56
   # no existe li.s
   # 0x40490E56 es la
   # representación IEEE754
   # en hexadecimal de 3.1415
fmv.w.x ft0, t0  # ft0 ← t0
li      t1  4    # 4 en Ca2
fcvt.s.w ft1, t1  # 4 en ieee754
fmul.s   ft0, ft0, ft1
```



Instrucciones de comparación (registros en coma flotante)

► Simple precisión

- `feq.s rd, rs1, rs2` if (rs1 == rs2) rd = 1; else rd = 0
- `fle.s rd, rs1, rs2` if (rs1 <= rs2) rd = 1; else rd = 0
- `flt.s rd, rs1, rs2` if (rs1 < rs2) rd = 1; else rd = 0

► Doble precisión:

- `feq.d rd, rs1, rs2` if (rs1 == rs2) rd = 1; else rd = 0
- `fle.d rd, rs1, rs2` if (rs1 <= rs2) rd = 1; else rd = 0
- `flt.d rd, rs1, rs2` if (rs1 < rs2) rd = 1; else rd = 0

Definición de datos estáticos (en tiempo de compilación)

etiqueta (dirección)

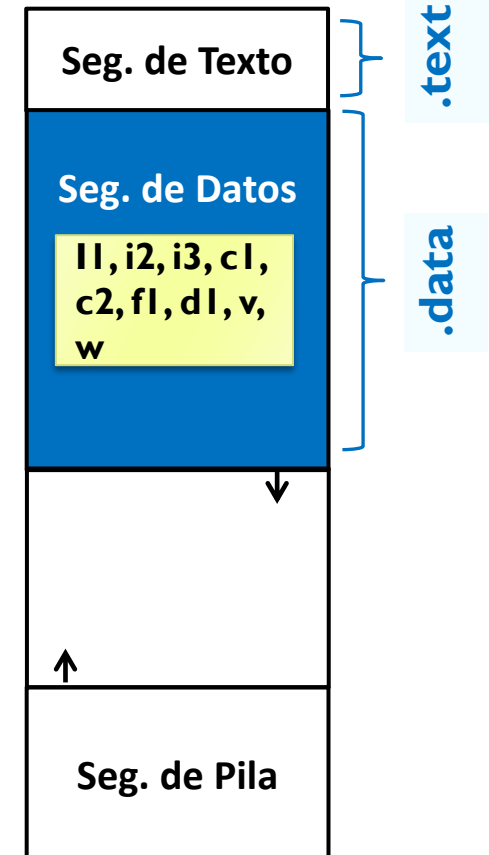
tipo de dato (directiva)

valor

```
.data
cadena .string "Hola mundo\n"
i1: .word 10      # int    i1=10
i2: .word -5      # int    i2=-5
i3: .half 300     # short  i3=300
c1: .byte 100     # char   c1=100
c2: .byte 'a'     # char   c2='a'
f1: .float 1.3e-4 # float  f1=1.3e-4
d1: .double .001  # double d1=0.001

# int v[3]={0,-1,0xffffffff}; int w[100];
v: .word 0, -1, 0xffffffff
w: .zero 400
```

0x00



Representación de tipos básicos (1/3)

```
// booleanos
bool_t b1 ;
bool_t b2 = false ;
```

```
// caracteres
char c1 ;
char c2 = 'x' ;
```

```
// enteros
int res1 ;
int op1 = -10 ;
```

```
// coma flotante
float f0 ;
float f1 = 1.2 ;
double d2 = 3.0e10 ;
```

```
.data

# boolean
b1:    .zero    1          # 1 bytes
b2:    .byte    0          # 1 byte

# character
c1:    .byte    'x'        # 1 bytes
c2:    .byte    'x'        # 1 bytes

# integers
res1:  .zero    4          # 4 bytes
op1:   .word    -10        # 4 bytes

# floating point
f0:    .float    0          # 4 bytes
f1:    .float    1.2        # 4 bytes
d2:    .double   3.0e10     # 8 bytes
```

Operaciones de memoria: coma flotante

Acceso a memoria (coma flotante)		
<code>flw rs, 10(rs1)</code>	<code>rs = Memoria[rs1+10]</code>	Carga el valor simple precisión almacenado en la dirección (rs1+10) en el registro de coma flotante rd.
<code>fsw rs, 10(rs1)</code>	<code>Memoria[rs1+10] = rs</code>	Almacena el valor simple precisión del registro rs en la dirección (rd1+10).
<code>fld rd, 10(rs1)</code>	<code>rd = Memoria[rs1+10]</code>	Carga el valor doble precisión almacenado en la dirección (rs1+10) en el registro rd.
<code>fsd rd, 10(rs1)</code>	<code>Memoria[rs1+10] = rd</code>	Almacena el valor doble precisión del registro rs en la dirección (rd1+10).

Tipo de datos básicos

float

```
float  resultado ;  
float  op1 = 100 ;  
float  op2 = 2.5  
...
```

```
main ()  
{  
    resultado = op1 + op2 ;  
    ...  
}
```

.data

.align 2

```
resultado:  .zero 4 # 4 bytes  
op1:        .float 100  
op2:        .float 2.5
```

...

.text

```
main: flw      ft0 op1(x0)  
      flw      ft1 op2(x0)  
      fadd.s   ft3 ft1 ft2  
      fsw      ft3 resultado(x0)  
      ...
```

Tipo de datos básicos

double

```
double resultado ;  
double op1 = 100 ;  
double op2 = -10.27 ;  
...
```

```
main ()  
{  
    resultado = op1 + op2 ;  
    ...  
}
```

.data

.align 3

```
resultado:  .zero 8  
op1:        .double 100  
op2:        .double -10.27
```

...

.text

```
main: fld      ft0 op1(x0)  
      fld      ft1 op2(x0)  
      fadd.d   ft3 ft1 ft2  
      fsd      ft3 resultado(x0)  
      ...
```

Bifurcaciones con números en coma flotante

Saltar a etiqueta si
ft1 < ft2

```
        flt t0, ft1, ft2
        bne t0, x0, etiqueta
        . . .
etiqueta:
```

Grupo ARCOS

uc3m | Universidad **Carlos III** de Madrid

Tema 3: Fundamentos de la programación en ensamblador (IV) **Estructura de Computadores**

Grado en Ingeniería Informática
Grado en Matemática aplicada y Computación
Doble Grado en Ingeniería Informática y Administración de Empresas



Ejemplo. Calcular

$$\sum_{i=1}^N \frac{1}{2^i}$$

```
.data
N:          .word 10          # Límite superior
one_f:      .float 1.0        # 1.0 en coma flotante
two_f:      .float 2.0        # 2.0 en coma flotante

.text

main:
    la      t0, N
    lw      t0, 0(t0)         # t0 = N = 10

    la      t1, one_f
    flw     ft0, 0(t1)        # ft0 = 1.0 (numerador)

    la      t1, two_f
    flw     ft1, 0(t1)        # ft1 = 2.0 (base de la potencia)

    fmv.w.x fa0, zero         # acumula el resultado

    # Inicializar variables del bucle
    li      t1, 1             # t1 = i = 1
    fmv.s   ft2, ft1          # ft2 = 2.0 (representará 2^i, empieza en 2^1)
```

Ejemplo. Calcular

$$\sum_{i=1}^N \frac{1}{2^i}$$

loop:

```
bgt    t1, t0, end_loop
```

```
# Calcular término actual: ft3 = 1.0 / (2^i)
```

```
fdiv.s  ft3, ft0, ft2      # ft3 = ft0 / ft2
```

```
# Sumar al acumulador total: fa0 = fa0 + ft3
```

```
fadd.s  fa0, fa0, ft3
```

```
# Actualizar para la siguiente iteración
```

```
fmul.s  ft2, ft2, ft1      # Multiplica la potencia por 2 para la sig. potencia (2^(i+1))
```

```
addi    t1, t1, 1          # i = i + 1
```

```
j       loop
```

end_loop:

```
#imprime el resultado
```

```
li a7, 2
```

```
ecall
```

```
li      a7, 10             # Syscall 10 = exit
```

```
ecall
```